A Gentle First Guide to Sparse Recovery, Lasso, and ISTA

May 26, 2026

Contents

1 Why these notes exist **4**

2 What is a numerical optimization problem? **4**

2.1 A one-variable example 5

2.2 A vector example 5

2.3 What does numerical mean? 5

3 Gradient descent from calculus **6**

3.1 The one-dimensional idea 6

3.2 A tiny worked example 6

3.3 The multivariable idea 7

3.4 Why the negative gradient decreases the function 7

4 Linear inverse problems and least squares **8**

4.1 Interpreting the dimensions 8

4.2 The gradient of least squares 9

5 Why ordinary least squares can be unstable **9**

5.1 Problem 1: too few measurements 10

5.2 Problem 2: noise amplification 10

6 A fast, friendly introduction to the SVD **10**

6.1 Why small singular values are dangerous 11

7 Regularization: adding a preference **11**

8 Ridge regression: the ℓ_2 penalty **12**

8.1 Solving ridge with calculus . . . 12

8.2 What ridge does in SVD coordinates . . . 13

8.3 Why ridge usually does not make exact zeros . . . 13

9 The goal of sparsity **14**

10 Why ℓ_1 produces sparse solutions **14**

10.1 A one-dimensional comparison . . . 14

10.2 The geometry in two dimensions . . . 15

11 The obstacle: ℓ_1 is not differentiable at zero **16**

12 Subgradients: slopes at a corner **16**

12.1 Why the interval $[-1, 1]$ makes sense . . . 17

12.2 Optimality for lasso . . . 17

13 The proximal idea **18**

13.1 The proximal operator as a compromise . . . 18

14 Soft-thresholding **19**

14.1 Deriving soft-thresholding from one-variable calculus . . . 19

14.2 Soft-thresholding a vector . . . 20

15 ISTA: gradient step plus soft-thresholding **21**

15.1 Pseudocode . . . 21

15.2 How to choose the step size . . . 22

15.3 How fast does ISTA converge? . . . 22

16 The companion numerical experiment **22**

16.1 The mathematical problem solved by the code . . . 23

16.2 The dimensions . . . 23

16.3 Building the true sparse vector . . . 23

16.4 Building the measurement matrix . . . 24

16.5 Adding noise . . . 24

16.6 Choosing the step size . . . 24
16.7 Choosing lambda . . . 24
16.8 The soft-thresholding line . . . 25
16.9 Experiment 1: sparse recovery . . . 25
16.10 Experiment 2: convergence . . . 26
16.11 Experiment 3: the regularization path . . . 26
16.12 The local ISTA function . . . 27

17 From sparse recovery to sparse discovery **27**

18 A compact summary of the whole story **28**

19 Exercises for students **29**

20 Short answer key **30**

21 Glossary **31**

22 Recommended next reading **31**

1 Why these notes exist

Many modern research problems have the following flavor. We collect data. We believe the data were produced by a simple hidden structure. We want a computer to discover that structure.

For example, suppose a physical system is governed by only a few important terms, but we do not know which terms. We might build a large list of candidate terms and ask the computer to select a small number of them. That word "small" is the doorway into sparsity.

A vector is called *sparse* when most of its entries are zero. A sparse vector is not merely a vector with small entries. It is a vector in which many entries are exactly zero.

Here is a toy sparse vector in $\mathbb{R}^{10}$:

$$x = (0, 0, 3.1, 0, 0, -1.4, 0, 0, 0, 2.2)^T.$$

Only three entries are nonzero. If the entries represented candidate scientific effects, then this vector would say: "Only effects 3, 6, and 10 seem active."

Big idea

Sparse recovery asks us to find a vector with many zeros from indirect measurements. Lasso and ISTA are a penalty and an algorithm for doing this.

The notes move in the following order.

1. We define what a numerical optimization problem is.
2. We derive gradient descent from calculus.
3. We introduce least squares problems of the form $Ax \approx b$.
4. We learn just enough about the singular value decomposition, or SVD, to understand why least squares can be unstable.
5. We study ridge regression, which uses an ℓ_2 penalty and improves stability.
6. We study lasso, which uses an ℓ_1 penalty and encourages exact zeros.
7. We introduce the soft-thresholding function.
8. We combine gradient descent with soft-thresholding to get ISTA.
9. We explain the companion MATLAB numerical experiment line by line at the conceptual level.

A few topics are deliberately treated as optional advanced background. For example, the precise connection between penalized and constrained forms of regularization uses Lagrange multipliers and convex duality. Those ideas are important, but they are not needed in order to understand the main algorithm. Standard references are Boyd and Vandenberghe (2004) for convex optimization and Nocedal and Wright (2006) for numerical optimization.

2 What is a numerical optimization problem?

An optimization problem asks us to choose the best object from a set of possible objects.

In this course, the object will usually be a vector

$$x = (x_1, x_2, \ldots, x_n)^T \in \mathbb{R}^n.$$

The function we want to make small is called the *objective function*, *loss function*, or *cost function*. We will often call it F.

The most basic form is

$$\min_{x \in \mathbb{R}^n} F(x).$$

Read this as:

> Find the vector x that makes $F(x)$ as small as possible.

The symbol min means "minimum." The expression underneath it tells us what we are allowed to choose. The function $F(x)$ tells us the score of our choice.

2.1 A one-variable example

Consider

$$\min_{x \in \mathbb{R}} (x - 3)^2.$$

The function $F(x) = (x-3)^2$ is zero when $x = 3$, and it is positive everywhere else. Therefore the answer is

$$x^\star = 3.$$

The star in $x^\star$ is a common notation for an optimal solution.

This example is simple enough to solve by looking. Most research problems are not.

2.2 A vector example

Now consider

$$\min_{x \in \mathbb{R}^2} (x_1 - 1)^2 + (x_2 + 2)^2.$$

The answer is

$$x^\star = (1, -2)^T.$$

Again, we can solve this by inspection.

But the problems in data science and scientific computing often have hundreds, thousands, or millions of variables. We cannot solve them by staring at the formula. That is why we use *numerical methods*.

2.3 What does numerical mean?

A numerical method is a recipe that a computer can follow using arithmetic. Instead of producing an exact answer in one magical step, a numerical optimization method usually produces a sequence of guesses:

$$x^0, x^1, x^2, x^3, \ldots.$$

The superscript is not an exponent. It is an iteration counter. The vector x^0 is the initial guess. The vector x^1 is the next guess. The vector x^k is the guess after k steps.

A good optimization algorithm tries to make

$$F(x^{k+1}) < F(x^k)$$

most of the time, or at least eventually. In words: each new guess should have a smaller objective value.

Checkpoint

When you see $\min_x F(x)$, ask yourself three questions.

1. What is the variable x?
2. What does the objective $F(x)$ measure?
3. Why would minimizing F solve the scientific or data problem?

3 Gradient descent from calculus

Before we can understand ISTA, we need to understand gradient descent. Gradient descent is the basic template behind many optimization algorithms.

3.1 The one-dimensional idea

Suppose $F : \mathbb{R} \to \mathbb{R}$ is differentiable. If we are currently at the point x, calculus says that for a small change h,

$$F(x + h) \approx F(x) + F'(x)h.$$

This is the linear approximation from first-semester calculus.

If $F'(x) > 0$, then the function is increasing as x moves to the right. To make F smaller, we should move left. If $F'(x) < 0$, then the function is decreasing as x moves to the right. To make F smaller, we should move right.

In both cases, the direction that decreases the function is the opposite of the derivative. So we update

$$x^{k+1} = x^k - tF'(x^k),$$

where $t > 0$ is called the *step size* or *learning rate*.

The number t controls how far we move. If t is too small, the method crawls. If t is too large, the method can jump past the minimum and even diverge.

3.2 A tiny worked example

Let

$$F(x) = \frac{1}{2}(x - 3)^2.$$

Then

$$F'(x) = x - 3.$$

Gradient descent gives

$$x^{k+1} = x^k - t(x^k - 3).$$

If $t = 1$, then

$$x^{k+1} = x^k - (x^k - 3) = 3.$$

In one step we land exactly at the minimizer. If $t = 0.2$, then

$$x^{k+1} = 0.8x^k + 0.6,$$

which moves more slowly toward 3.

3.3 The multivariable idea

Now suppose $F : \mathbb{R}^n \to \mathbb{R}$. The derivative becomes the gradient

$$\nabla F(x) = \begin{bmatrix} \frac{\partial F}{\partial x_1}(x) \\ \frac{\partial F}{\partial x_2}(x) \\ \vdots \\ \frac{\partial F}{\partial x_n}(x) \end{bmatrix}.$$

The gradient points in the direction of steepest increase. Therefore, the negative gradient points in the direction of steepest local decrease.

The gradient descent update is

$$x^{k+1} = x^k - t\nabla F(x^k).$$

This is the same formula as in one variable, except x and $\nabla F(x)$ are now vectors.

Big idea

Gradient descent is local. At the current point, it asks: "Which way does the function increase fastest?" Then it moves in the opposite direction.

3.4 Why the negative gradient decreases the function

The multivariable linear approximation says

$$F(x + p) \approx F(x) + \nabla F(x)^T p,$$

where p is a proposed step direction. If we choose

$$p = -t\nabla F(x),$$

then

$$\nabla F(x)^T p = \nabla F(x)^T \left(-t\nabla F(x)\right) = -t \left\|\nabla F(x)\right\|_2^2.$$

This number is negative unless $\nabla F(x) = 0$. Therefore, for a small enough step size t, moving in the negative gradient direction should decrease the objective.

This is the core calculus idea behind gradient descent.

Careful

The phrase "small enough" matters. A direction can be correct, but a step can still be too long. Choosing good step sizes is one of the central practical issues in numerical optimization.

4 Linear inverse problems and least squares

Now we introduce the problem family that will lead to lasso and ISTA.

Suppose we have a matrix $A \in \mathbb{R}^{m \times n}$, a data vector $b \in \mathbb{R}^m$, and an unknown vector $x \in \mathbb{R}^n$. We want

$$Ax \approx b.$$

The matrix A maps possible unknown vectors x into predicted measurements Ax. The vector b contains the measurements we actually observed.

The difference

$$r(x) = Ax - b$$

is called the *residual*. It measures the mismatch between prediction and data.

A natural way to measure the size of the residual is with the Euclidean norm:

$$\|Ax - b\|_2 = \sqrt{(Ax-b)_1^2 + \cdots + (Ax-b)_m^2}.$$

Because square roots are inconvenient and do not change where the minimum occurs, we usually minimize the squared residual:

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - b\|_2^2 .$$

The factor $1/2$ is included only because it makes derivatives cleaner.

This is the least squares problem.

4.1 Interpreting the dimensions

The matrix A has m rows and n columns.

- m is the number of measurements.
- n is the number of unknowns.

If $m > n$, we have more equations than unknowns. This is called an overdetermined problem.

If $m < n$, we have fewer equations than unknowns. This is called an underdetermined problem.

Sparse recovery often lives in the underdetermined case. We may have fewer measurements than unknowns, but we hope the true unknown vector is sparse.

4.2 The gradient of least squares

Let

$$f(x) = \frac{1}{2} \|Ax - b\|_2^2 .$$

We will need its gradient. The result is

$$\nabla f(x) = A^T(Ax - b).$$

Let us derive this carefully.

Set $r = Ax - b$. If we perturb x by a small vector h, then

$$A(x + h) - b = Ax - b + Ah = r + Ah.$$

Therefore

$$\begin{aligned} f(x + h) &= \frac{1}{2} \|r + Ah\|_2^2 \\ &= \frac{1}{2}(r + Ah)^T(r + Ah) \\ &= \frac{1}{2}r^T r + r^T Ah + \frac{1}{2}h^T A^T Ah. \end{aligned}$$

The first term is $f(x)$. The last term is quadratic in h, so it is smaller than first-order terms when h is tiny. The first-order term is

$$r^T Ah.$$

We rewrite it so that h is on the right:

$$r^T Ah = (A^T r)^T h.$$

By the definition of the gradient, the vector multiplying h is the gradient. Thus

$$\nabla f(x) = A^T r = A^T(Ax - b).$$

Checkpoint

Every time ISTA computes

$$A^T(Ax - b),$$

it is computing the gradient of the smooth least squares part.

5 Why ordinary least squares can be unstable

If least squares is so natural, why not stop there?

There are two common problems.

5.1 Problem 1: too few measurements

If $m < n$, then there are more unknowns than equations. There may be infinitely many vectors x that fit the data equally well.

For example, one equation in two unknowns,

$$x_1 + x_2 = 1,$$

has infinitely many solutions:

$$(1, 0), \quad (0, 1), \quad (2, -1), \quad (100, -99), \quad \dots$$

A computer needs a rule for choosing among them.

5.2 Problem 2: noise amplification

Even if the least squares solution is unique, it may be very sensitive to small changes in the data b.

Suppose a small measurement error changes b by a small vector e. If the computed solution changes by a huge amount, the problem is unstable. This can happen when the matrix A has directions that it nearly crushes to zero.

The singular value decomposition gives us the cleanest way to understand this.

6 A fast, friendly introduction to the SVD

The singular value decomposition, or SVD, is one of the most useful tools in numerical linear algebra. A standard reference is Trefethen and III (1997); a more introductory reference is Strang (2016).

The SVD says that any matrix $A \in \mathbb{R}^{m \times n}$ can be written as

$$A = U \Sigma V^T.$$

This formula looks intimidating at first, but the meaning is simple:

> A matrix transformation can be understood as a rotation/reflection, then stretching, then another rotation/reflection.

Here is the interpretation.

- V^T changes coordinates in the input space.
- Σ stretches or shrinks along coordinate axes.
- U changes coordinates in the output space.

The diagonal entries of Σ are called the *singular values*:

$$\sigma_1 \geq \sigma_2 \geq \cdots \geq 0.$$

Large singular values stretch. Small singular values shrink. A zero singular value completely destroys information in some direction.

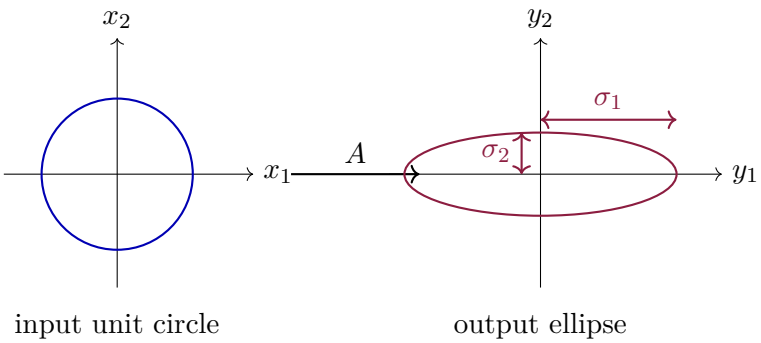


Figure 1: A matrix maps the unit circle to an ellipse. The lengths of the principal axes of that ellipse are the singular values.

6.1 Why small singular values are dangerous

Imagine A shrinks one direction by a factor of 0.001. Then two very different input vectors may produce almost the same output. If we try to reverse the process, we must divide by 0.001, which multiplies errors by 1000.

This is the main reason small singular values cause instability.

For a full-rank least squares problem, the solution can be written in SVD coordinates as

$$x_{\text{LS}} = \sum_i \frac{u_i^T b}{\sigma_i} v_i.$$

The vectors u_i and v_i are columns of U and V. The important part is the division by σ_i.

If σ_i is tiny, then the coefficient $(u_i^T b)/\sigma_i$ can become huge, even when $u_i^T b$ is not huge. This is noise amplification.

Big idea

Least squares can be unstable because solving backwards through A requires division by singular values. Tiny singular values cause huge divisions.

7 Regularization: adding a preference

When a problem is unstable or has many solutions, we add an extra term that expresses a preference. This is called *regularization*.

The regularized least squares problem is

$$\min_{x \in \mathbb{R}^n} \frac{1}{2} \|Ax - b\|_2^2 + \lambda R(x).$$

Here:

- $\frac{1}{2} \|Ax - b\|_2^2$ is the data-fit term.

- $R(x)$ is the regularization penalty.
- $\lambda > 0$ controls the strength of the penalty.

The parameter λ is pronounced "lambda." It is a tuning knob.

If λ is small, the algorithm cares mostly about fitting the data. If λ is large, the algorithm cares more about satisfying the preference encoded by $R(x)$.

Different choices of $R(x)$ produce different behavior. We will compare the two most important choices:

$$R(x) = \frac{1}{2}\|x\|_2^2 \quad \text{and} \quad R(x) = \|x\|_1 .$$

8 Ridge regression: the ℓ_2 penalty

The ℓ_2 norm is

$$\|x\|_2 = \sqrt{x_1^2 + x_2^2 + \cdots + x_n^2}.$$

Ridge regression uses the squared ℓ_2 penalty:

$$\min_x \frac{1}{2}\|Ax - b\|_2^2 + \frac{\lambda}{2}\|x\|_2^2 .$$

The factor $1/2$ again makes derivatives cleaner.

8.1 Solving ridge with calculus

Define

$$F_{\text{ridge}}(x) = \frac{1}{2}\|Ax - b\|_2^2 + \frac{\lambda}{2}\|x\|_2^2 .$$

We already know

$$\nabla\left(\frac{1}{2}\|Ax - b\|_2^2\right) = A^T(Ax - b).$$

Also,

$$\nabla\left(\frac{\lambda}{2}\|x\|_2^2\right) = \lambda x.$$

Therefore

$$\nabla F_{\text{ridge}}(x) = A^T(Ax - b) + \lambda x.$$

At a minimizer, the gradient is zero:

$$A^T(Ax - b) + \lambda x = 0.$$

Rearrange:

$$A^T Ax - A^T b + \lambda x = 0,$$

so

$$(A^T A + \lambda I)x = A^T b.$$

If $\lambda > 0$, the matrix $A^T A + \lambda I$ is much better behaved than $A^T A$. The ridge solution is

$$x_\lambda = (A^T A + \lambda I)^{-1} A^T b.$$

8.2 What ridge does in SVD coordinates

Using the SVD, ridge can be written as

$$x_\lambda = \sum_i \frac{\sigma_i}{\sigma_i^2 + \lambda}(u_i^T b)v_i.$$

A more revealing way to write the same expression is

$$x_\lambda = \sum_i \underbrace{\frac{\sigma_i^2}{\sigma_i^2 + \lambda}}_{\text{filter factor}} \frac{u_i^T b}{\sigma_i} v_i.$$

The term

$$\frac{\sigma_i^2}{\sigma_i^2 + \lambda}$$

is between 0 and 1. It is called a filter factor.

If σ_i^2 is much larger than λ, then

$$\frac{\sigma_i^2}{\sigma_i^2 + \lambda} \approx 1.$$

The corresponding component mostly passes through.

If σ_i^2 is much smaller than λ, then

$$\frac{\sigma_i^2}{\sigma_i^2 + \lambda} \approx 0.$$

The corresponding component is strongly damped.

Big idea

Ridge regression protects us from tiny singular values. It does not fully trust directions where A nearly destroys information.

8.3 Why ridge usually does not make exact zeros

Ridge shrinks entries toward zero, but it usually does not set entries exactly equal to zero.

A simple one-dimensional example explains why. Consider

$$\min_x \frac{1}{2}(x - z)^2 + \frac{\lambda}{2}x^2.$$

Take the derivative:

$$(x - z) + \lambda x = 0.$$

So

$$x = \frac{z}{1 + \lambda}.$$

Unless $z = 0$, this answer is not zero. It is just smaller than z.

This is the essence of ridge: smooth shrinkage, not exact selection.

9 The goal of sparsity

In sparse recovery, we want the solution itself to choose only a few coordinates.

A natural penalty would be

$$\|x\|_0 = \text{number of nonzero entries of } x.$$

This is called the ℓ_0 "norm," though it is not truly a norm. It directly counts how many entries are active.

For example,

$$x = (0, 5, 0, -2, 0)^T$$

has

$$\|x\|_0 = 2.$$

The problem is that minimizing with an ℓ_0 penalty is generally combinatorial. The computer may have to search over many possible subsets of coordinates.

The lasso replaces the hard-to-optimize ℓ_0 count with the easier ℓ_1 norm:

$$\|x\|_1 = |x_1| + |x_2| + \cdots + |x_n| .$$

The lasso problem is

$$\min_x F(x) = \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1 .$$

Big idea

The ℓ_1 norm is not the same as counting nonzeros, but it often behaves like a computationally friendly substitute for counting nonzeros.

The lasso was introduced in statistics by Tibshirani (1996). It is now a standard tool across statistics, signal processing, imaging, inverse problems, and scientific machine learning.

10 Why ℓ_1 produces sparse solutions

The function $|x|$ has a corner at $x = 0$.

That corner is the whole story.

10.1 A one-dimensional comparison

Compare these two penalties:

$$\text{ridge penalty: } \frac{1}{2}x^2, \qquad \text{lasso penalty: } |x| .$$

The derivative of $\frac{1}{2}x^2$ is x. At $x = 0$, the derivative is 0. So the penalty becomes flat at the origin.

The function $|x|$ has slope -1 to the left of zero and slope $+1$ to the right of zero. At zero, it has a sharp corner.

That sharp corner makes it possible for the optimum to stick exactly at zero.

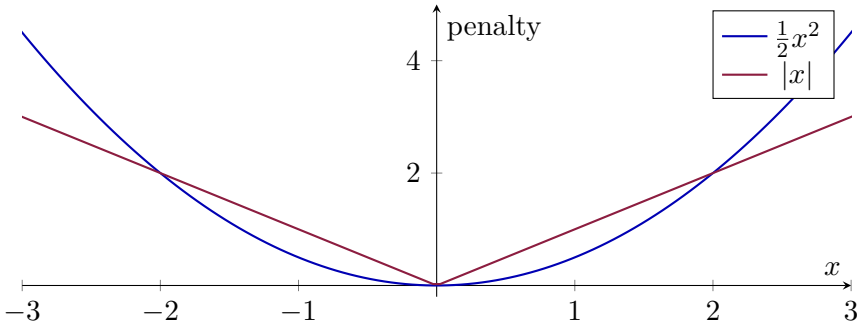


Figure 2: The ℓ_2 penalty is smooth at zero. The ℓ_1 penalty has a corner at zero.

10.2 The geometry in two dimensions

In two dimensions,

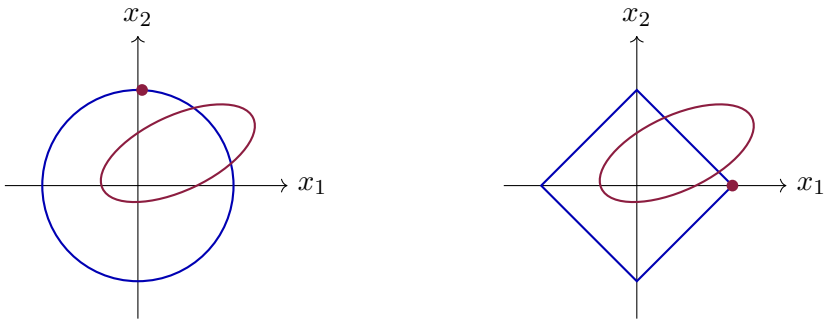
$$\|x\|_2 \leq \tau$$

is a disk, while

$$\|x\|_1 \leq \tau$$

is a diamond.

The disk is round. The diamond has corners on the coordinate axes.



ℓ_2 ball: round boundary ℓ_1 ball: corners on axes

Figure 3: A round constraint tends to touch a contour away from the axes. A diamond-shaped constraint often touches at a corner, where one coordinate is exactly zero.

The picture is often explained using the constrained form

$$\min_x \frac{1}{2}\|Ax - b\|_2^2 \quad \text{subject to} \quad \|x\|_p \leq \tau.$$

For $p = 2$, the constraint is round. For $p = 1$, the constraint has corners.

As the level curves of the least squares term expand, the first touching point is likely to hit a corner of the ℓ_1 diamond. At a corner, one coordinate is exactly zero.

Optional advanced note

The penalized lasso form

$$\frac{1}{2}\|Ax-b\|_2^2+\lambda\|x\|_1$$

and the constrained form

$$\|x\|_1 \le \tau$$

are closely related. Making this relationship precise uses Lagrange multipliers and convex duality. This is standard material in convex optimization; see Boyd and Vandenberghe (2004). For the present notes, the constrained picture is used as geometric intuition, not as a required proof.

11 The obstacle: ℓ_1 is not differentiable at zero

Gradient descent needs a gradient. But the lasso objective is

$$F(x)=\frac{1}{2}\|Ax-b\|_2^2+\lambda\|x\|_1.$$

The least squares part is differentiable. The ℓ_1 part is not differentiable when any coordinate $x_i=0$.

This is not a small technical annoyance. The whole point of lasso is to produce zeros, exactly where ordinary derivatives fail.

So we need a replacement idea.

There are two levels of replacement:

1. Subgradients explain what optimality means at a corner.
2. Proximal operators lead to a practical algorithm.

We will keep both ideas as concrete as possible.

12 Subgradients: slopes at a corner

The absolute value function has no single derivative at 0.

For $x>0$, the derivative of $|x|$ is 1. For $x<0$, the derivative of $|x|$ is -1. At $x=0$, the left slope and right slope do not match.

Instead of choosing one slope, convex analysis allows a whole interval of slopes:

$$\partial|x|=\begin{cases}\{1\}, & x>0,\\ [-1,1], & x=0,\\ \{-1\}, & x<0.\end{cases}$$

The symbol ∂ means subdifferential. At smooth points, it contains one number. At a corner, it contains many possible slopes.

12.1 Why the interval $[-1, 1]$ makes sense

At $x = 0$, the graph of $|x|$ has many supporting lines. A supporting line touches the graph and stays below it.

Lines through the origin with slope between -1 and 1 all stay below $|x|$. That is why every slope in $[-1, 1]$ is allowed.

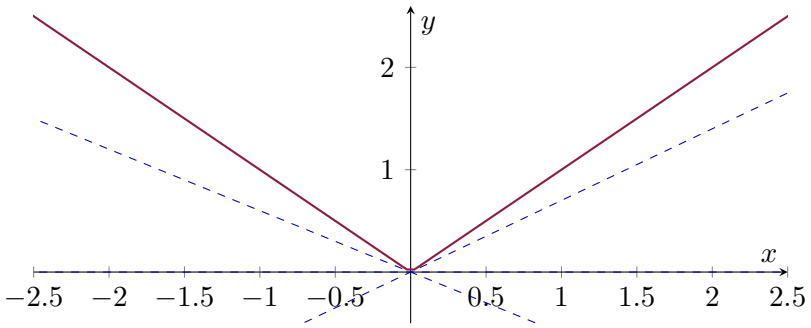


Figure 4: At the corner of $|x|$, many lines can support the graph from below. Their slopes fill the interval $[-1, 1]$.

12.2 Optimality for lasso

For a differentiable convex function, the minimizer satisfies

$$\nabla F(x^\star) = 0.$$

For a nonsmooth convex function, the analogous condition is

$$0 \in \partial F(x^\star).$$

Read this as: zero must be one of the allowed slopes.

For the lasso, the condition becomes

$$0 \in A^T(Ax^\star - b) + \lambda \partial \left\|x^\star\right\|_1 .$$

Let

$$r = b - Ax^\star$$

be the residual. Let a_i be column i of A. Then the lasso optimality condition says:

$$a_i^T r = \lambda \operatorname{sign}(x_i^\star) \quad \text{if } x_i^\star \neq 0,$$

and

$$\left|a_i^T r\right| \leq \lambda \quad \text{if } x_i^\star = 0.$$

This condition has an important interpretation. A coordinate is allowed to remain zero when its column of A is not correlated enough with the current residual. The threshold is λ.

Big idea

Lasso keeps a coordinate at zero when that coordinate does not help explain the remaining residual strongly enough to overcome the ℓ_1 penalty.

13 The proximal idea

Now we move from optimality conditions to an algorithm.

The lasso objective is the sum of two pieces:

$$F(x) = f(x) + g(x),$$

where

$$f(x) = \frac{1}{2}\|Ax - b\|_2^2, \qquad g(x) = \lambda \|x\|_1 .$$

The function f is smooth. The function g is nonsmooth but simple.

Proximal gradient descent says:

> Take a gradient step on the smooth part, then apply a special correction for the nonsmooth part.

The special correction is called a proximal operator.

13.1 The proximal operator as a compromise

For a function g, the proximal operator is

$$\text{prox}_{tg}(z) = \arg\min_x \left\{ \frac{1}{2t}\|x - z\|_2^2 + g(x) \right\}.$$

This definition is easier to understand in words.

Given a point z, the proximal operator chooses a new point x that balances two desires:

- Stay close to z, because of $\frac{1}{2t}\|x - z\|_2^2$.
- Make the penalty $g(x)$ small.

The parameter t controls how strongly we care about staying close to z.

For lasso, the magic is that the proximal operator has a very simple formula.

14 Soft-thresholding

The soft-thresholding function is

$$\mathcal{S}_\eta(z) = \operatorname{sign}(z)\max\{|z| - \eta, 0\}.$$

Here $\eta \geq 0$ is the threshold.

In plain language:

- If z is positive and larger than η, subtract η.
- If z is negative and smaller than $-\eta$, add η.
- If z is between $-\eta$ and η, replace it by 0.

Equivalently,

$$\mathcal{S}_\eta(z) = \begin{cases} z - \eta, & z > \eta, \\ 0, & -\eta \leq z \leq \eta, \\ z + \eta, & z < -\eta. \end{cases}$$

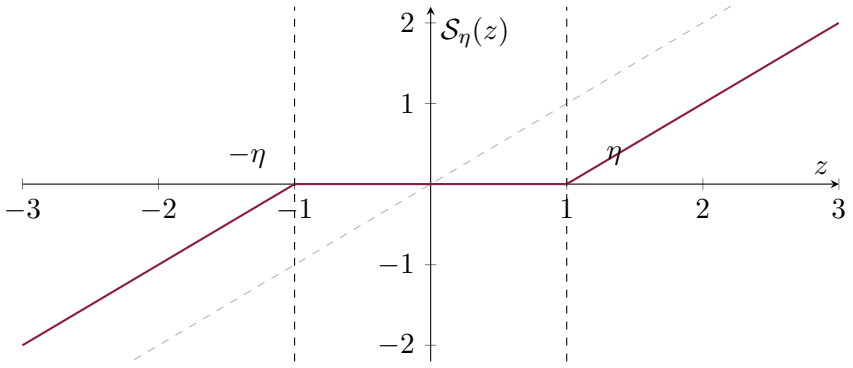


Figure 5: Soft-thresholding shrinks values toward zero and clamps small values exactly to zero. Here $\eta = 1$.

14.1 Deriving soft-thresholding from one-variable calculus

We now derive the formula. This is the most important calculation in the notes.

Consider the scalar problem

$$\min_x \ \frac{1}{2}(x - z)^2 + \eta\,|x|\,.$$

This is the one-dimensional proximal problem for the ℓ_1 norm. We want to find the best x for a given z.

Because of $|x|$, we split into three cases.

Case 1: $x > 0$. If $x > 0$, then $|x| = x$. The objective becomes

$$\frac{1}{2}(x - z)^2 + \eta x.$$

Differentiate:

$$(x - z) + \eta = 0.$$

So

$$x = z - \eta.$$

This case is valid only if $x > 0$, meaning $z - \eta > 0$, or $z > \eta$.

Case 2: $x < 0$**.** If $x < 0$, then $|x| = -x$. The objective becomes

$$\frac{1}{2}(x - z)^2 - \eta x.$$

Differentiate:

$$(x - z) - \eta = 0.$$

So

$$x = z + \eta.$$

This case is valid only if $x < 0$, meaning $z + \eta < 0$, or $z < -\eta$.

Case 3: $x = 0$**.** The remaining possibility is that the minimizer occurs at the corner. This happens when z is not large enough in magnitude to escape the penalty. The condition is

$$-\eta \leq z \leq \eta.$$

In this case the solution is $x = 0$.

Putting the three cases together gives

$$\arg\min_x \left\{ \frac{1}{2}(x - z)^2 + \eta \, |x| \right\} = \mathcal{S}_\eta(z).$$

Big idea

Soft-thresholding is the exact solution of a tiny optimization problem involving the absolute value penalty. It is not a heuristic trick.

14.2 Soft-thresholding a vector

The ℓ_1 norm is separable:

$$\|x\|_1 = |x_1| + |x_2| + \cdots + |x_n| \, .$$

Because each coordinate appears separately, the proximal operator acts coordinate by coordinate:

$$\text{prox}_{\eta\|\cdot\|_1}(z) = \begin{bmatrix} \mathcal{S}_\eta(z_1) \\ \mathcal{S}_\eta(z_2) \\ \vdots \\ \mathcal{S}_\eta(z_n) \end{bmatrix}.$$

This coordinatewise behavior is one reason lasso is computationally friendly.

15 ISTA: gradient step plus soft-thresholding

We are finally ready for ISTA.

Recall the lasso objective:

$$F(x) = \frac{1}{2}\|Ax - b\|_2^2 + \lambda \|x\|_1 .$$

Let

$$f(x) = \frac{1}{2}\|Ax - b\|_2^2 , \qquad g(x) = \lambda \|x\|_1 .$$

We know

$$\nabla f(x) = A^T(Ax - b).$$

Proximal gradient descent does two things.

First, take a gradient step on f:

$$z^k = x^k - tA^T(Ax^k - b).$$

Second, apply the proximal operator of g:

$$x^{k+1} = \text{prox}_{t\lambda\|\cdot\|_1}(z^k).$$

Since the prox is soft-thresholding, we get

$$x^{k+1} = \mathcal{S}_{t\lambda}\left(x^k - tA^T(Ax^k - b)\right).$$

This is ISTA.

ISTA in words

At each iteration:

1. Compute the residual $Ax^k - b$.
2. Move in the negative gradient direction $-A^T(Ax^k - b)$.
3. Soft-threshold the result so that small coordinates become exactly zero.

15.1 Pseudocode

ISTA pseudocode

1. Choose an initial guess x^0, often $x^0 = 0$.
2. Choose a step size t.
3. For $k = 0, 1, 2, \ldots$:
$$z^k = x^k - tA^T(Ax^k - b),$$
$$x^{k+1} = \mathcal{S}_{t\lambda}(z^k).$$
4. Stop when the changes are small, or after a fixed number of iterations.

15.2 How to choose the step size

For the least squares function

$$f(x) = \frac{1}{2}\|Ax - b\|_2^2,$$

the gradient is

$$\nabla f(x) = A^T(Ax - b).$$

This gradient changes at a rate controlled by

$$L = \left\|A^T A\right\|_2 = \|A\|_2^2 = \sigma_{\max}(A)^2.$$

Here $\|A\|_2$ is the spectral norm, which equals the largest singular value of A.

A safe fixed step size is

$$t = \frac{1}{L}.$$

This is exactly what the companion code uses:

```
L = norm(A)^2;
t = 1 / L;
```

Optional advanced note

The statement "$t \leq 1/L$ guarantees convergence" is part of the standard theory of proximal gradient methods. A friendly modern treatment is Beck (2017); a classical numerical optimization reference is Nocedal and Wright (2006).

15.3 How fast does ISTA converge?

Let $x^\star$ be a lasso minimizer. Under standard assumptions, ISTA with $t = 1/L$ satisfies

$$F(x^k) - F(x^\star) \leq \frac{L\|x^0 - x^\star\|_2^2}{2k}.$$

The main message is

$$F(x^k) - F(x^\star) = O(1/k).$$

This means the objective gap decreases roughly like a constant divided by the iteration number.

There is an accelerated version called FISTA, which adds a momentum step and obtains a faster $O(1/k^2)$ rate in objective value. The soft-thresholding step is the same. FISTA was developed in this setting by Beck and Teboulle (2009); related iterative thresholding ideas appear in Daubechies et al. (2004).

16 The companion numerical experiment

The MATLAB file demonstrates the whole story on a synthetic sparse recovery problem.

The problem is synthetic because the code creates the true answer first. This is useful for learning: since we know the true sparse vector, we can check whether ISTA recovers it.

16.1 The mathematical problem solved by the code

The code solves

$$\min_x \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1 .$$

The data are generated by

$$b = Ax_{\text{true}} + \varepsilon,$$

where

- x_{true} is a sparse vector,
- A is a random measurement matrix,
- ε is small measurement noise.

The code then asks: from only A and b, can we recover something close to x_{true}?

16.2 The dimensions

The script sets

```
n = 200;       % number of unknowns
m = 80;        % number of measurements, m < n
s = 8;         % number of nonzero entries in the ground truth
sigma = 0.02;  % measurement noise level
```

So the unknown vector has length 200, but we only have 80 measurements. This is underdetermined.

Ordinary least squares alone cannot identify a unique meaningful vector without extra assumptions. The extra assumption is sparsity: the true vector has only 8 nonzero entries.

16.3 Building the true sparse vector

The code uses

```
xtrue = zeros(n, 1);
support = randperm(n, s);
xtrue(support) = randn(s, 1);
```

This creates a vector of length $n = 200$ filled with zeros. Then it randomly selects $s = 8$ locations. Those locations are called the *support*. Finally, it places random nonzero values at those locations.

For example, if the support were

$$\{4, 19, 73, 101, 120, 144, 155, 190\},$$

then only those eight entries of x_{true} would be nonzero.

Checkpoint

The support of a vector is the set of indices where the vector is nonzero.

16.4 Building the measurement matrix

The code uses

```
A = randn(m, n) / sqrt(m);
```

This creates an 80×200 random matrix. Dividing by $\sqrt{m}$ keeps the columns at a reasonable scale. Without this scaling, changing the number of measurements would also change the typical size of matrix-vector products.

16.5 Adding noise

The code uses

```
b = A * xtrue + sigma * randn(m, 1);
```

This means

$$b = Ax_{\text{true}} + \varepsilon.$$

The term σ `randn(m,1)` is random noise. Since $\sigma = 0.02$, the noise is small but not zero.

This matters. In real experiments, measurements are almost never perfect.

16.6 Choosing the step size

The code uses

```
L = norm(A)^2;
t = 1 / L;
```

In MATLAB, `norm(A)` is the spectral norm by default. That is the largest singular value of A. Therefore `norm(A)^2` is

$$L = \sigma_{\max}(A)^2.$$

The step size is $t = 1/L$, the standard safe choice for ISTA.

16.7 Choosing lambda

The code uses

```
lambda_max = max(abs(A' * b));
lambda = 0.1 * lambda_max;
```

The number

$$\lambda_{\max} = \max_i \left| a_i^T b \right|$$

is the smallest value of λ for which the all-zero vector is optimal.

Why? At $x = 0$, the residual in the lasso optimality condition is $r = b$. The condition for coordinate i to remain zero is

$$\left| a_i^T b \right| \leq \lambda.$$

This must hold for every i. Therefore we need

$$\lambda \geq \max_i \left| a_i^T b \right|.$$

The code chooses

$$\lambda = 0.1\lambda_{\max},$$

which is smaller than the all-zero threshold. This allows some coordinates to become active.

16.8 The soft-thresholding line

The code defines

```
soft = @(z, th) sign(z) .* max(abs(z) - th, 0);
```

This is exactly

$$\mathcal{S}_{\text{th}}(z) = \text{sign}(z) \max\{|z| - \text{th}, 0\}.$$

The `.*` means componentwise multiplication in MATLAB. Since z is a vector, the formula is applied to every coordinate.

16.9 Experiment 1: sparse recovery

The code runs

```
maxit = 2000;
[xhat, Fhist] = ista(A, b, lambda, t, maxit, soft);
```

This starts from $x^0 = 0$ and performs 2000 ISTA iterations.

The output `xhat` is the recovered sparse vector. The output `Fhist` stores the objective value at every iteration:

$$F(x^k) = \frac{1}{2} \left\| Ax^k - b \right\|_2^2 + \lambda \left\| x^k \right\|_1.$$

The script then plots both x_{true} and $\hat{x}$ using stem plots.

What should students look for?

- Are the largest recovered spikes near the true nonzero locations?
- Did ISTA set most coordinates to zero?
- Are there any false positives, meaning recovered nonzeros where the true vector is zero?
- Are there any missed entries, meaning true nonzeros that were not recovered?

Because the data include noise and because A is random, perfect recovery is not guaranteed in every possible run. But the experiment is designed so that the lasso solution should reveal the sparse structure.

16.10 Experiment 2: convergence

The code next studies whether ISTA behaves like the theory predicts.

ISTA has the theoretical objective-gap behavior

$$F(x^k) - F(x^\star) = O(1/k).$$

But in the experiment, we do not know the exact optimum $x^\star$. So the script approximates it by running ISTA for many more iterations:

```
[~, Flong] = ista(A, b, lambda, t, 20000, soft);
Fstar = Flong(end);
```

Then it computes

```
gap = Fhist - Fstar;
```

This approximates the objective gap

$$F(x^k) - F(x^\star).$$

The code plots the gap on a log-log scale and compares it with a curve proportional to $1/k$:

```
loglog(1:maxit, gap)
hold on
loglog(1:maxit, gap(1) ./ (1:maxit), '--')
```

On a log-log plot, a $1/k$ curve appears as a straight line with slope -1. If the ISTA gap roughly follows that reference line, the experiment matches the theory.

16.11 Experiment 3: the regularization path

The third experiment varies λ.

The code creates 60 values from large to small:

```
nl = 60;
lambdas = logspace(log10(lambda_max), log10(0.01 * lambda_max), nl);
```

When λ is large, the penalty is strong. The lasso solution is very sparse, often exactly zero.

When λ is small, the penalty is weaker. More coordinates are allowed to enter the solution.

For each value of λ, the code runs ISTA and stores the solution:

```
Xpath = zeros(n, nl);
xwarm = zeros(n, 1);
for k = 1:nl
    xwarm = ista_warm(A, b, lambdas(k), t, 500, soft, xwarm);
    Xpath(:, k) = xwarm;
end
```

The variable `xwarm` is a warm start. This means that the solution for one value of λ is used as the initial guess for the next value. Warm starts are useful because nearby λ values often have nearby solutions.

The final plot shows each coordinate of x_λ as a function of λ. This is called the regularization path.

Students should notice that as λ decreases, coordinates switch on. This is the dynamic version of sparsity.

16.12 The local ISTA function

The core MATLAB function is short:

```
function [x, Fhist] = ista(A, b, lambda, t, maxit, soft)
    x = zeros(size(A, 2), 1);
    Fhist = zeros(maxit, 1);
    for k = 1:maxit
        grad = A' * (A * x - b);
        x = soft(x - t * grad, t * lambda);
        Fhist(k) = 0.5 * norm(A * x - b)^2 + lambda * sum(abs(x));
    end
end
```

Each line corresponds directly to the mathematics:

$$\texttt{grad = A' * (A * x - b)} \quad \leftrightarrow \quad \nabla f(x) = A^T(Ax - b),$$

$$\texttt{x = soft(x - t * grad, t * lambda)} \quad \leftrightarrow \quad x^{k+1} = \mathcal{S}_{t\lambda}\left(x^k - t\nabla f(x^k)\right).$$

Big idea

The whole advanced algorithm is hiding in two lines: compute the least squares gradient, then soft-threshold.

17 From sparse recovery to sparse discovery

The companion code is a sparse recovery experiment. Sparse discovery uses the same mathematical pattern in a scientific modeling setting.

A common sparse discovery setup looks like this:

$$\Theta\xi \approx y.$$

Here:

- Θ is a library matrix whose columns are candidate features or candidate terms.
- y is a measured quantity we want to explain, such as a time derivative.
- ξ is a coefficient vector.

If only a few candidate terms are truly active, then ξ should be sparse.

The lasso-style problem becomes

$$\min_{\xi} \frac{1}{2} \|\Theta\xi - y\|_2^2 + \lambda \|\xi\|_1 .$$

This has the same structure as

$$\min_{x} \frac{1}{2} \|Ax - b\|_2^2 + \lambda \|x\|_1 .$$

Only the names have changed.

This is why learning lasso and ISTA is useful for sparse discovery. The algorithmic core transfers directly.

For a well-known sparse discovery method for dynamical systems, see the SINDy work of Brunton et al. (2016). For compressed sensing foundations, see Candes et al. (2006) and Donoho (2006).

18 A compact summary of the whole story

We started with a linear model

$$Ax \approx b.$$

Least squares solves

$$\min_{x} \frac{1}{2} \|Ax - b\|_2^2 .$$

This can be unstable or nonunique, especially when there are too few measurements or when A has tiny singular values.

Regularization adds a preference:

$$\min_{x} \frac{1}{2} \|Ax - b\|_2^2 + \lambda R(x).$$

Ridge uses

$$R(x) = \frac{1}{2} \|x\|_2^2 .$$

It improves stability by damping small-singular-value directions. It shrinks coefficients smoothly, but usually does not set them exactly to zero.

Lasso uses

$$R(x) = \|x\|_1 .$$

The ℓ_1 norm has corners. Those corners encourage exact zeros. This is why lasso is useful for sparse recovery.

Because ℓ_1 is not differentiable at zero, plain gradient descent is not the right tool. Proximal gradient descent fixes this by taking a gradient step on the smooth least squares term and then applying the proximal operator of the ℓ_1 norm.

That proximal operator is soft-thresholding:

$$\mathcal{S}_\eta(z) = \text{sign}(z) \max\{|z| - \eta, 0\}.$$

Therefore ISTA is

$$x^{k+1} = \mathcal{S}_{t\lambda}\left(x^k - tA^T(Ax^k - b)\right).$$

The companion code demonstrates this algorithm by recovering a sparse vector from noisy underdetermined measurements, plotting the convergence rate, and tracing the lasso regularization path.

19 Exercises for students

Exercise 1: Reading an optimization problem

For each problem, identify the variable and the objective function.

1. $\min_x (x-5)^2$.
2. $\min_{w \in \mathbb{R}^3} \|Bw - c\|_2^2$.
3. $\min_z \frac{1}{2}\|Mz - y\|_2^2 + 0.1\|z\|_1$.

Exercise 2: One step of gradient descent

Let

$$F(x) = \frac{1}{2}(x-4)^2.$$

Start at $x^0 = 0$ and use step size $t = 0.25$. Compute x^1, x^2, x^3.

Exercise 3: Gradient of least squares

Let

$$A = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}, \qquad b = \begin{bmatrix} 1 \\ 3 \end{bmatrix}, \qquad x = \begin{bmatrix} 2 \\ 1 \end{bmatrix}.$$

Compute $Ax - b$ and then compute $A^T(Ax - b)$.

Exercise 4: Soft-thresholding by hand

Let $\eta = 0.5$. Compute

$$\mathcal{S}_\eta(2), \quad \mathcal{S}_\eta(0.2), \quad \mathcal{S}_\eta(-3), \quad \mathcal{S}_\eta(-0.4).$$

Exercise 5: Ridge shrinkage in one dimension

Solve

$$\min_x \frac{1}{2}(x-z)^2 + \frac{\lambda}{2}x^2.$$

Show that the answer is $x = z/(1+\lambda)$.

Exercise 6: Lasso shrinkage in one dimension

Solve

$$\min_x \frac{1}{2}(x-z)^2 + \eta\,|x|$$

by splitting into the cases $x > 0$, $x < 0$, and $x = 0$. Recover the soft-thresholding formula.

Exercise 7: Change the numerical experiment

Run the companion MATLAB script several times, changing one parameter at a time.

1. Increase the noise level `sigma`. What happens to support recovery?
2. Increase the sparsity level `s`. What happens when the true signal is less sparse?
3. Increase the number of measurements `m`. Does recovery improve?
4. Change `lambda = 0.1 * lambda_max` to `lambda = 0.5 * lambda_max`. Does the solution become more or less sparse?

20 Short answer key

Exercise 2

The update is

$$x^{k+1} = x^k - 0.25(x^k - 4) = 0.75x^k + 1.$$

Starting at $x^0 = 0$:

$$x^1 = 1, \qquad x^2 = 1.75, \qquad x^3 = 2.3125.$$

Exercise 3

First,

$$Ax = \begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix} \begin{bmatrix} 2 \\ 1 \end{bmatrix} = \begin{bmatrix} 4 \\ 1 \end{bmatrix}.$$

So

$$Ax - b = \begin{bmatrix} 4 \\ 1 \end{bmatrix} - \begin{bmatrix} 1 \\ 3 \end{bmatrix} = \begin{bmatrix} 3 \\ -2 \end{bmatrix}.$$

Then

$$A^T(Ax - b) = \begin{bmatrix} 1 & 0 \\ 2 & 1 \end{bmatrix} \begin{bmatrix} 3 \\ -2 \end{bmatrix} = \begin{bmatrix} 3 \\ 4 \end{bmatrix}.$$

Exercise 4

With $\eta = 0.5$:

$$\mathcal{S}_{0.5}(2) = 1.5, \quad \mathcal{S}_{0.5}(0.2) = 0, \quad \mathcal{S}_{0.5}(-3) = -2.5, \quad \mathcal{S}_{0.5}(-0.4) = 0.$$

21 Glossary

Objective function
: The function being minimized.

Optimization variable
: The unknown quantity we are allowed to choose.

Residual
: The mismatch $Ax - b$ between prediction and measurement.

Least squares
: Minimizing the squared residual norm $\frac{1}{2}\|Ax - b\|_2^2$.

Regularization
: Adding a penalty to stabilize a problem or encode a preference.

Ridge regression
: Regularized least squares with an ℓ_2 squared penalty.

Lasso
: Regularized least squares with an ℓ_1 penalty.

Sparse vector
: A vector with many zero entries.

Support
: The set of indices where a vector is nonzero.

SVD
: A factorization $A = U\Sigma V^T$ that reveals how a matrix stretches and shrinks directions.

Singular value
: A nonnegative number describing how much a matrix stretches or shrinks a special direction.

Subgradient
: A generalized slope for a convex function that may have corners.

Proximal operator
: A map that balances staying close to a point with reducing a penalty.

Soft-thresholding
: The proximal operator of the ℓ_1 norm; it shrinks entries and sets small entries to zero.

ISTA
: Iterative Shrinkage-Thresholding Algorithm; proximal gradient descent applied to lasso.

Regularization path
: The collection of solutions obtained as λ varies.

22 Recommended next reading

For students who want more linear algebra, start with Strang (2016), then move to Trefethen and III (1997). For numerical optimization, use Nocedal and Wright (2006). For convex optimization, including subgradients and duality, use Boyd and Vandenberghe (2004). For proximal algorithms and convergence rates, use Beck (2017). For the statistical lasso viewpoint, see Tibshirani (1996) and Hastie et al. (2009). For sparse recovery and compressed sensing, see Candes et al. (2006) and Donoho (2006). For sparse discovery of governing equations, see Brunton et al. (2016).

References

Beck, Amir (2017). *First-Order Methods in Optimization*. SIAM.

Beck, Amir and Marc Teboulle (2009). "A Fast Iterative Shrinkage-Thresholding Algorithm for Linear Inverse Problems". In: *SIAM Journal on Imaging Sciences* 2.1, pp. 183–202. DOI: 10.1137/080716542.

Boyd, Stephen and Lieven Vandenberghe (2004). *Convex Optimization*. Cambridge University Press.

Brunton, Steven L., Joshua L. Proctor, and J. Nathan Kutz (2016). "Discovering Governing Equations from Data by Sparse Identification of Nonlinear Dynamical Systems". In: *Proceedings of the National Academy of Sciences* 113.15, pp. 3932–3937. DOI: 10.1073/pnas.1517384113.

Candes, Emmanuel J., Justin Romberg, and Terence Tao (2006). "Robust Uncertainty Principles: Exact Signal Reconstruction from Highly Incomplete Frequency Information". In: *IEEE Transactions on Information Theory* 52.2, pp. 489–509. DOI: 10.1109/TIT.2005.862083.

Daubechies, Ingrid, Michel Defrise, and Christine De Mol (2004). "An Iterative Thresholding Algorithm for Linear Inverse Problems with a Sparsity Constraint". In: *Communications on Pure and Applied Mathematics* 57.11, pp. 1413–1457. DOI: 10.1002/cpa.20042.

Donoho, David L. (2006). "Compressed Sensing". In: *IEEE Transactions on Information Theory* 52.4, pp. 1289–1306. DOI: 10.1109/TIT.2006.871582.

Hastie, Trevor, Robert Tibshirani, and Jerome Friedman (2009). *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. 2nd ed. Springer.

Nocedal, Jorge and Stephen J. Wright (2006). *Numerical Optimization*. 2nd ed. Springer.

Strang, Gilbert (2016). *Introduction to Linear Algebra*. 5th ed. Wellesley-Cambridge Press.

Tibshirani, Robert (1996). "Regression Shrinkage and Selection via the Lasso". In: *Journal of the Royal Statistical Society: Series B* 58.1, pp. 267–288.

Trefethen, Lloyd N. and David Bau III (1997). *Numerical Linear Algebra*. SIAM.